OWASP JUICE SHOP

Web Application Security Assessment

Penetration Test Report

Academic Cybersecurity Project

Date: 14 March 2026

Target: http://localhost:3000 (OWASP Juice Shop v19.1.1)

Classification: Academic TEK-UP Project

Elaborated by: Mohamed Taher BORGI

Class: 4-J-SSIR-I

Confidentiality Statement

This document is the exclusive property of **Mohamed Taher BORGI** and TEK-UP.

This document contains proprietary and confidential information. Duplication, redistribution, or use, in whole or in part, in any form, requires prior consent.

Disclaimer

This report was produced as part of an academic penetration testing exercise. The target application is **OWASP Juice Shop**, an intentionally vulnerable web application designed for security training.

All testing was conducted in an isolated, controlled environment on a locally hosted instance.

No real systems, user data, or production infrastructure were accessed or harmed during this assessment.

The findings and techniques described herein are intended solely for educational purposes.

This document must not be used to attack systems without explicit written authorization.

Table of Contents

1. Assessment Overview
2. Information Gathering & Technology Fingerprinting
3. Executive Summary
4. Vulnerability Summary & Report Card
5. Technical Findings
   WA-001: Exposed FTP Directory & Sensitive File Disclosure (Critical)
   WA-002: XML External Entity (XXE) Injection (Critical)
   WA-003: Broken Cryptographic Implementation (Critical)
   WA-004a: SQL Injection — Authentication Bypass (Critical)
   WA-004b: Insufficient Brute-Force Protection on Login Endpoint (High)
   WA-005: Reflected Cross-Site Scripting (XSS) (High)
   WA-006: Application-Level Denial of Service (High)
   WA-007: File Download Filter Bypass via Null Byte (Medium)
   WA-008: Unauthenticated Admin API Access (Medium)
   WA-009: Use of Components with Known Vulnerabilities (Low)

1. Assessment Overview

The objective of this assessment was to evaluate the security posture of the OWASP Juice Shop web application through a black-box penetration test. The methodology followed the standard phases outlined in OWASP Testing Guide v4 and PTES (Penetration Testing Execution Standard):

- Reconnaissance — Passive and active information gathering.
- Enumeration — Mapping attack surfaces, API endpoints, and technology stack.
- Exploitation — Confirming vulnerabilities through controlled proof-of-concept attacks.
- Post-Exploitation — Assessing the impact of successful compromises.
- Reporting — Documenting findings with evidence and remediation guidance.

Scope

Target	Details
Application	OWASP Juice Shop v19.1.1
URL	http://localhost:3000
Test Type	Black-box Web Application Penetration Test
Environment	Local (isolated lab environment)
Exclusions	Phishing, Social Engineering

2. Information Gathering & Technology Fingerprinting

To map the attack surface, automated fingerprinting was performed using **Wappalyzer** and manual **HTTP header** analysis. The application follows a modern decoupled architecture with a Single Page Application (SPA) frontend.

2.1 Technology Stack

Category	Technology	Version	Security Implication
Frontend Framework	Angular	20.3.17	Client-side routing; potential DOM-based XSS and template injection.
Backend Environment	Node.js	Detected	NoSQL Injection; Prototype Pollution risk.
JavaScript Library	jQuery	2.2.4	Legacy version. Known XSS vulnerabilities (CVE-2015-9251). OWASP A06:2021.
Language	TypeScript	Compiled	Source maps may expose original code in production.
CDN	Cloudflare / cdnjs	-	Check for missing Subresource Integrity (SRI) attributes.

Key Observations

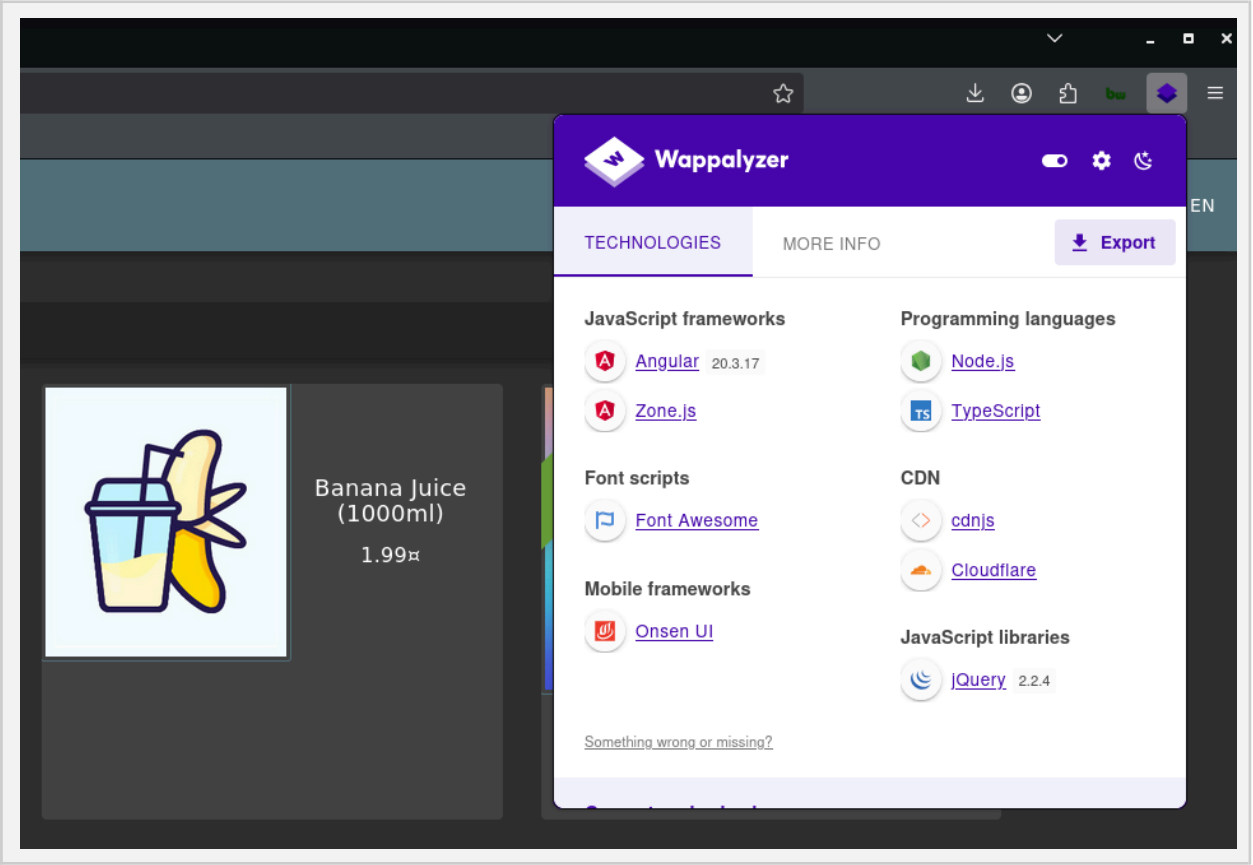
- Legacy Library Risk: jQuery 2.2.4 is susceptible to known XSS vulnerabilities (CVE-2015-9251) — flagged as OWASP A06:2021 (Vulnerable and Outdated Components).
- Decoupled Architecture: The Angular + Node.js stack confirms a RESTful API model. Security focus shifts to JWT handling, CORS configuration, and API endpoint authorization.
- Angular SPA routing uses hash-based paths (/#/route). This means server-side directory fuzzing alone cannot enumerate client-side routes.

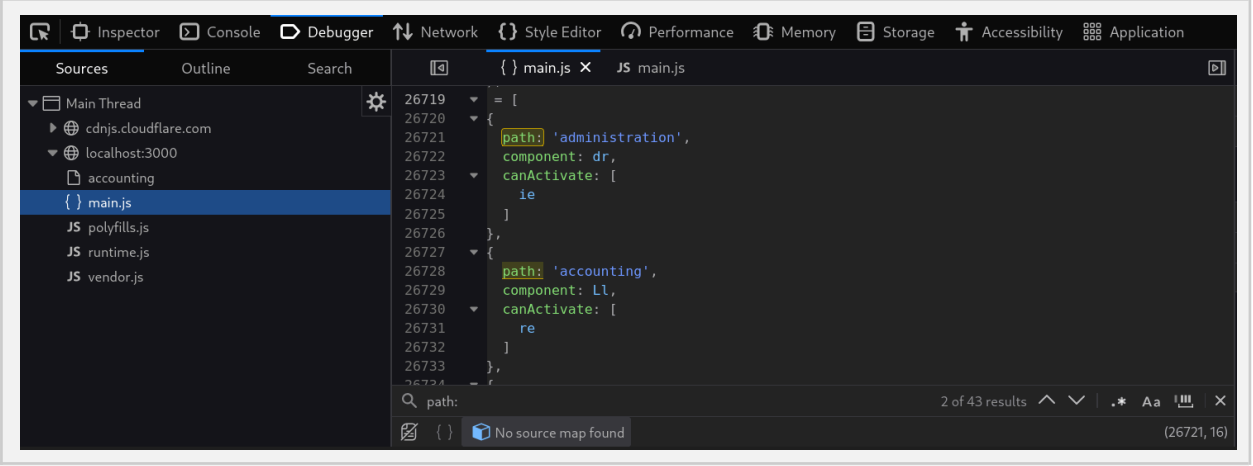
2.2 Enumeration Methodology

Two complementary techniques were used:

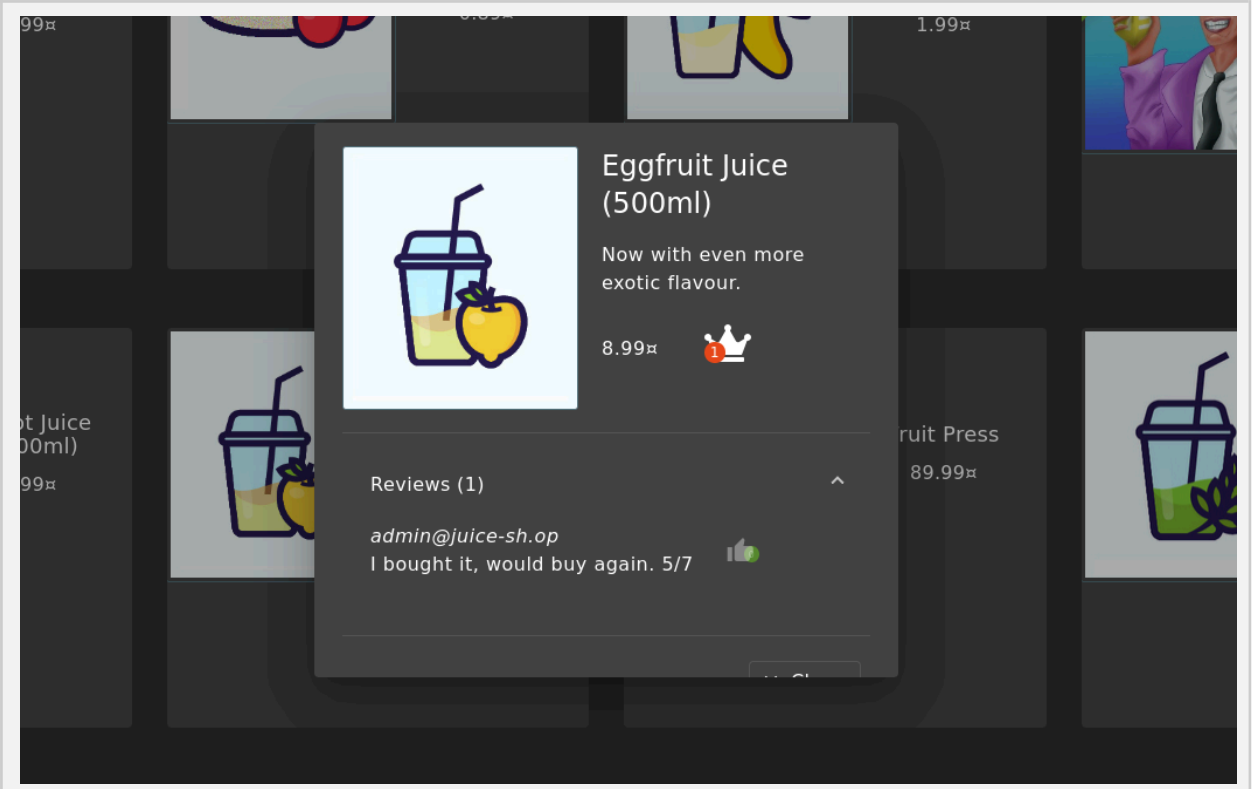
- **Client-Side Static Analysis**: Manual inspection of main.js (Angular bundle) revealed the full routing table, including the hidden /#/administration path.
- **Active Directory Brute-forcing**: ffuf was used with the common.txt wordlist to enumerate server-side paths. A wildcard routing baseline of 75,055 bytes was filtered to eliminate false positives.

Evidence Passive Enumeration :





Evidence of admin login address showing in Reviews section :



Evidence Active Enumeration :

```
[Status: 200, Size: 11297, Words: 1579, Lines: 358, Duration: 273ms]
| URL | http://localhost:3000/ftp
    * FUZZ: ftp

[Status: 200, Size: 450244, Words: 7200, Lines: 1, Duration: 215ms]
| URL | http://localhost:3000/main.js
    * FUZZ: main.js

[Status: 200, Size: 6586, Words: 560, Lines: 177, Duration: 277ms]
| URL | http://localhost:3000/promotion
    * FUZZ: promotion

[Status: 200, Size: 28, Words: 3, Lines: 2, Duration: 63ms]
| URL | http://localhost:3000/robots.txt
    * FUZZ: robots.txt

[Status: 200, Size: 28, Words: 3, Lines: 2, Duration: 64ms]
| URL | http://localhost:3000/robots.txt
    * FUZZ: robots.txt

[Status: 200, Size: 475, Words: 49, Lines: 7, Duration: 57ms]
| URL | http://localhost:3000/security.txt
    * FUZZ: security.txt
```

3. Executive Summary

The security assessment of the OWASP Juice Shop application revealed a total of 9 vulnerabilities, including 4 Critical and 2 High severity findings.

The overall security posture of the application is **poor**, reflecting multiple OWASP Top 10 categories.

The most severe attack path discovered allows an unauthenticated attacker to fully compromise the application through a chain of vulnerabilities:

- Step 1 — Reconnaissance: Technology fingerprinting identified an outdated jQuery version and a decoupled Angular/Node.js architecture, guiding subsequent attack vectors.
- Step 2 — Information Disclosure: The **/ftp** directory was found **publicly accessible**, exposing backup files, an encrypted password vault, and server-side dependency manifests.
- Step 3 — File Exfiltration: A **Null Byte bypass (%2500.md)** allowed restricted binary files to be downloaded, bypassing the server-side file extension whitelist.
- Step 4 — Cryptographic Attack: Decompilation of encrypt.pyc exposed a broken RSA implementation, enabling **full decryption** of encrypted announcements.
- Step 5 — Authentication Bypass: Both a **SQL injection** payload (`' OR 1=1--`) and a credential **brute-force attack** (ffuf + fasttrack.txt) independently granted administrative access, confirming the login endpoint has no injection protection and no rate limiting.
- Step 6 — **Full Compromise**: With **administrator** access, all user data, orders, and application settings were accessible.

Key Recommendations

Three immediate priorities were identified:

- Disable unauthenticated access to sensitive directories (/ftp) and administrative APIs.
- Implement parameterized queries across all database interactions to eliminate SQL injection.
- Configure XML parsers to reject DTDs and external entity declarations.
- Implement rate limiting and account lockout on the login endpoint to prevent credential brute-forcing.

4. Vulnerability Summary & Report Card

4	3	2	0	1
Critical	High	Medium	Low	Informational

Finding	Severity	Recommendation
WA-001	**Critical**	Disable /ftp directory listing; remove sensitive files from web root.
WA-002	**Critical**	Configure XML parser to reject DTDs and external entities.
WA-003	**Critical**	Replace custom RSA implementation with RSA-OAEP standard library.
WA-004a	**Critical**	Use parameterized queries to prevent SQL injection.
WA-004b	**High**	Implement account lockout and rate limiting on the login endpoint.
WA-005	**High**	Implement context-aware output encoding; apply strict Content Security Policy.
WA-006	**High**	Implement rate limiting (express-rate-limit); use a process manager (PM2).
WA-007	**Medium**	Sanitize file download parameters; block Null Byte sequences.
WA-008	**Medium**	Restrict /rest/admin/* to authenticated, authorized sessions only.
WA-009	**Low**	Upgrade jQuery to a supported version; harden HTTP response headers.

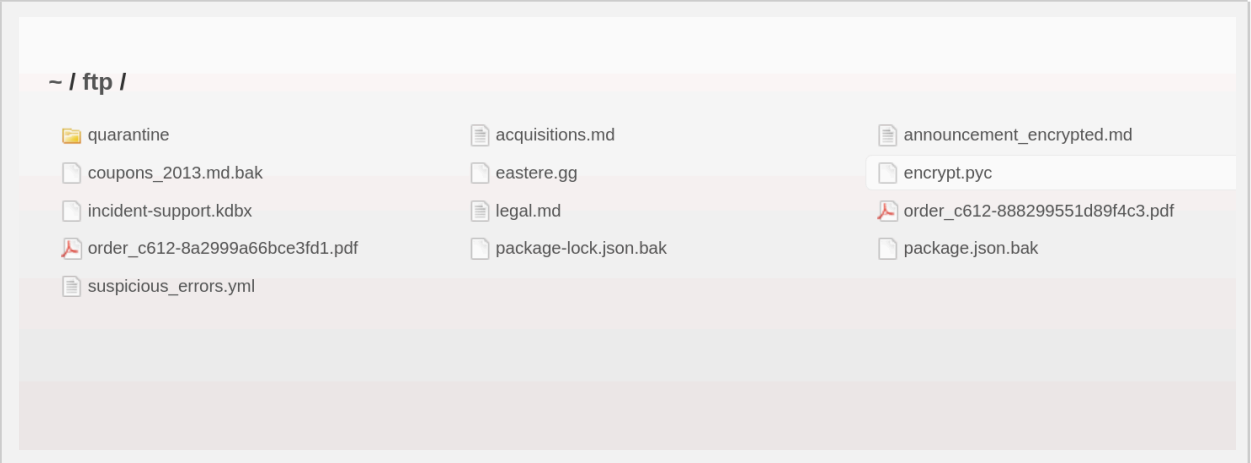
*WA = Web Application

5. Technical Findings

Finding WA-001: Exposed FTP Directory & Sensitive File Disclosure

Severity	Critical
Description	The /ftp directory is publicly accessible without authentication and directory listing is enabled. The robots.txt file explicitly references this path (Disallow: /ftp), confirming its existence. The directory contains highly sensitive files including a KeePass password vault (incident-support.kdbx), compiled Python encryption logic (encrypt.pyc), internal acquisition documents, application backup files (package.json.bak), and a suspicious_errors.yml log file. The package.json.bak file reveals server-side dependencies including sqlite3 ~3.1.13 and sanitize-html 1.4.2, providing direct exploit leads.
System	http://localhost:3000/ftp/*
Tools Used	Browser, curl, robots.txt, ffuf
References	OWASP A01:2021 – Broken Access Control; CWE-548: Information Exposure Through Directory Listing

Evidence



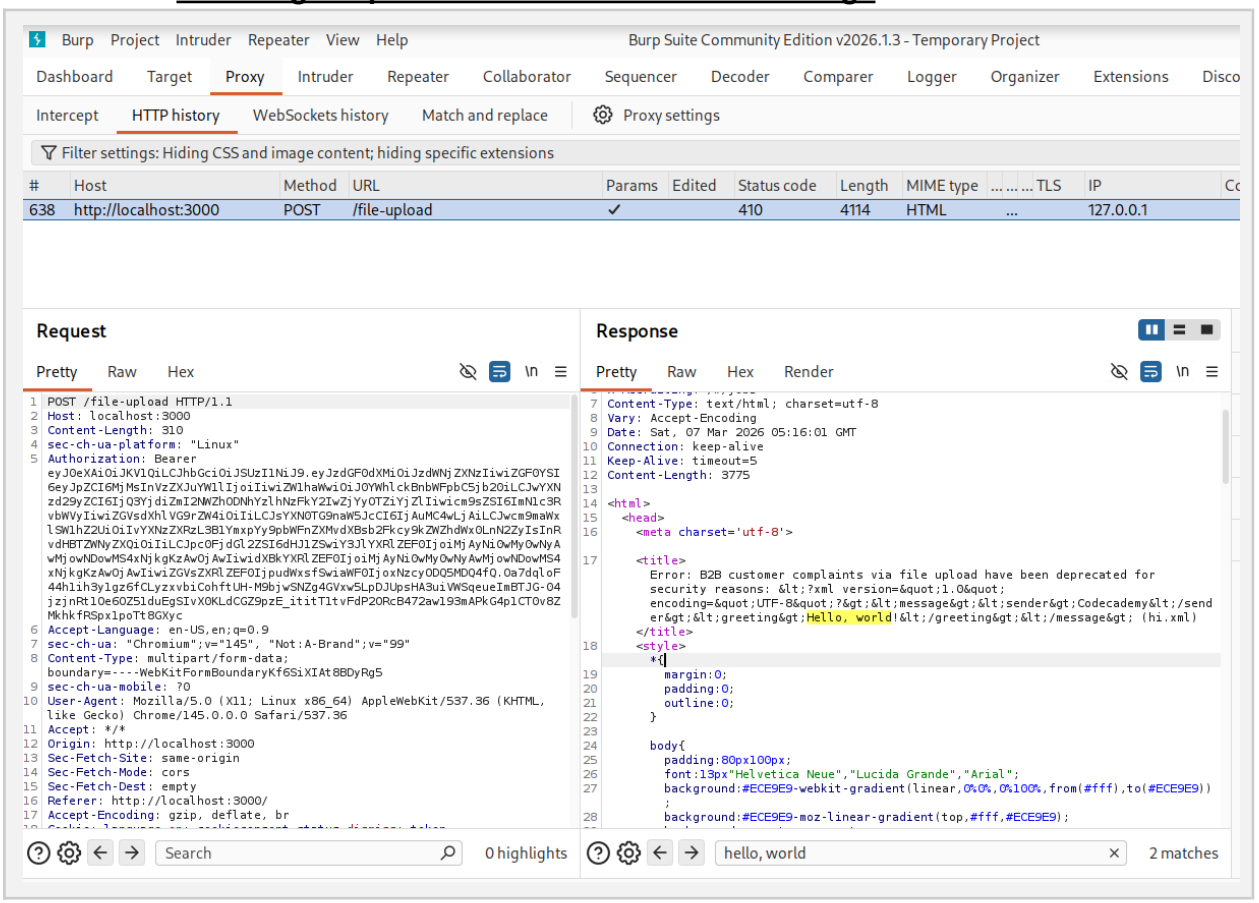
Remediation

Disable directory listing on the web server. Move all sensitive files out of the web root. Ensure robots.txt does not serve as a roadmap for attackers. Restrict /ftp access behind authentication.

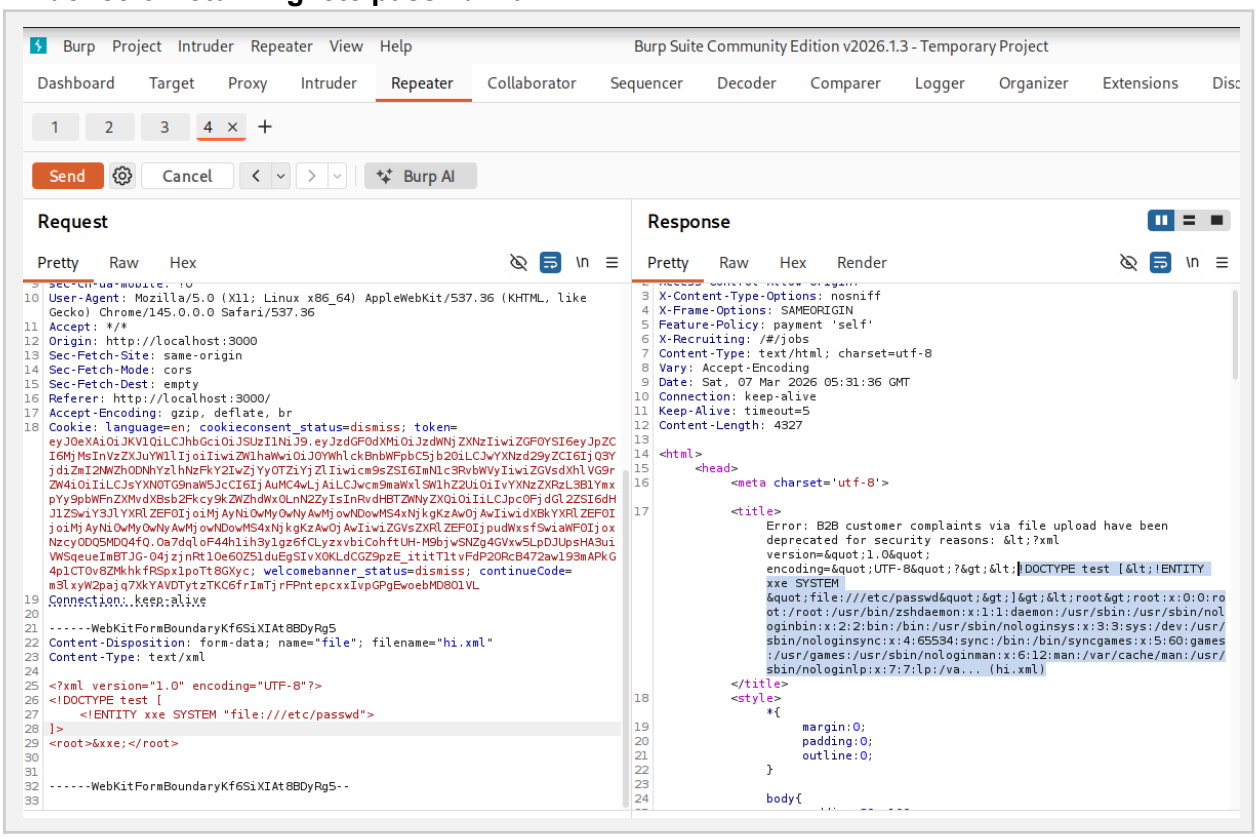
Finding WA-002: XML External Entity (XXE) Injection via B2B File Upload

Severity	Critical
Description	The deprecated B2B file upload endpoint (/file-upload) still processes uploaded XML files using an unconfigured parser. Despite returning HTTP 410 Gone, the parser resolves external entities and reflects the parsed content in the error message. By uploading an XML file containing a DTD that defines an external entity pointing to /etc/passwd, the contents of the system file were returned in the server error response. This constitutes an Error-Based XXE attack.
System	http://localhost:3000/file-upload
Tools Used	Burp Suite Repeater, custom XML payloads
References	CWE-611: Improper Restriction of XML External Entity Reference; OWASP A03:2021 – Injection

Evidence of reflecting the parsed content in the error message



Evidence of returning /etc/passwd via XXE



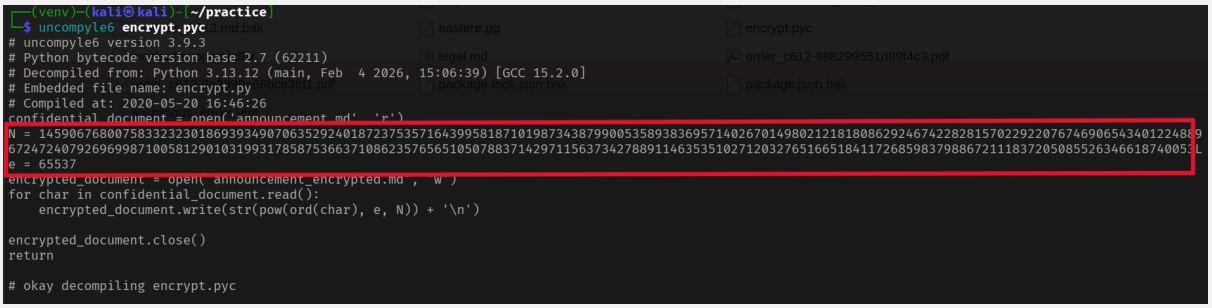
Remediation

Configure the XML parser to disable DTDs and external entity processing (set noent to false). Completely remove the deprecated upload route from the codebase. Implement generic error messages that do not reflect user-supplied input.

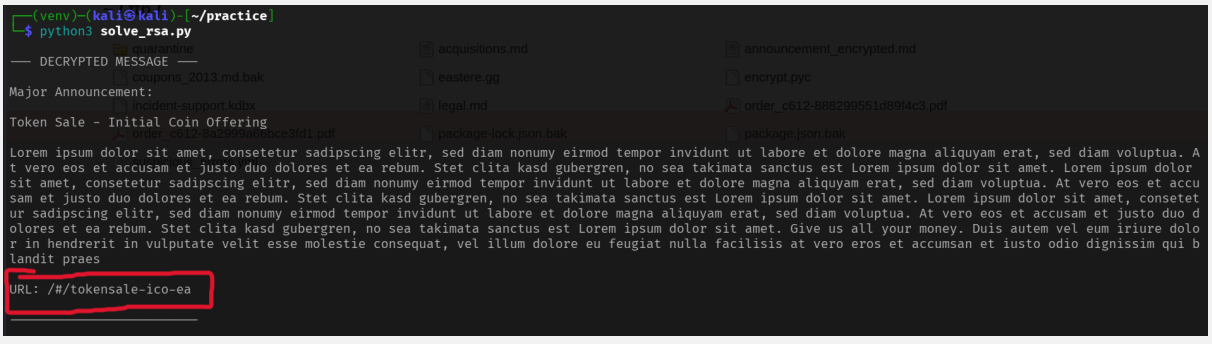
Finding WA-003: Broken Cryptographic Implementation

Severity	Critical
Description	The file encrypt.pyc (downloaded from /ftp via Null Byte bypass) was decompiled using uncompyle6, revealing a custom RSA implementation that encrypts data character-by-character without OAEP padding. Because no padding is used, encryption is fully deterministic: the same plaintext character always produces the same ciphertext value. This allows a Known-Plaintext / Lookup Table attack. A Python script pre-computed the RSA output for all 256 ASCII characters using the recovered modulus N and exponent e, then used the table to fully decrypt announcement_encrypted.md, revealing an unpublished ICO token sale announcement.
System	http://localhost:3000/ftp/encrypt.pyc (via bypass)
Tools Used	curl, uncompyle6, Python3
References	CWE-327: Use of a Broken or Risky Cryptographic Algorithm; OWASP A02:2021 – Cryptographic Failures

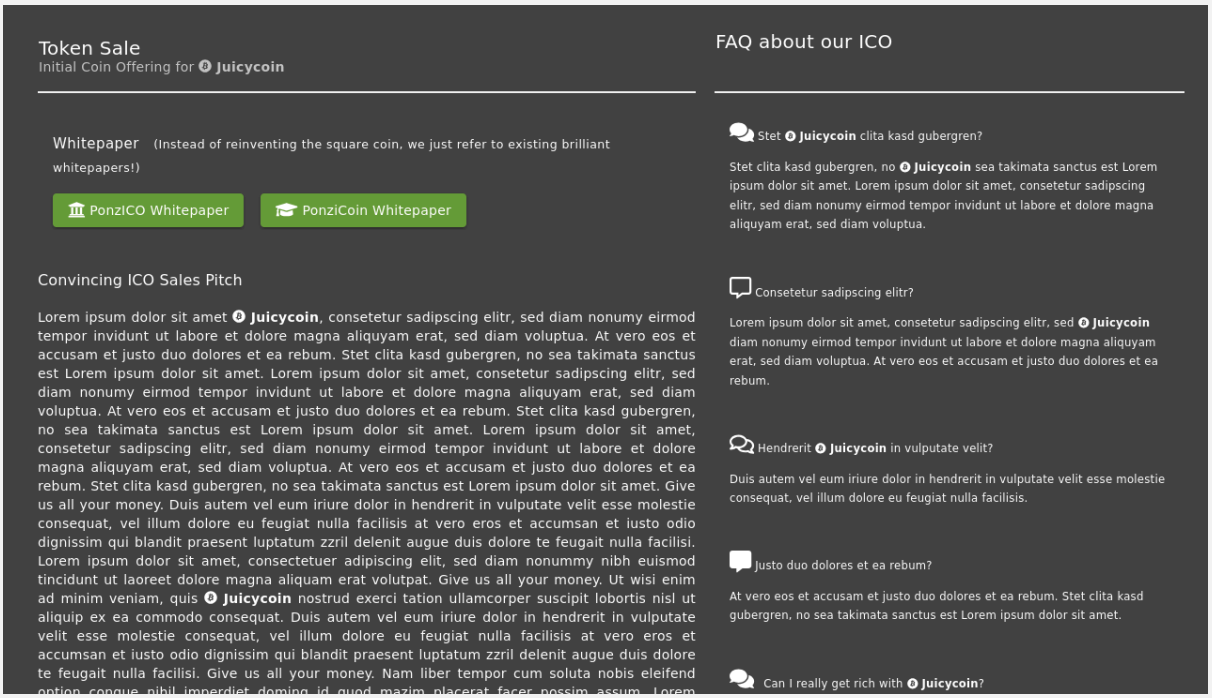
Evidence



Evidence of Decrypting the message using a Python Script



Evidence of accessing the non published Report



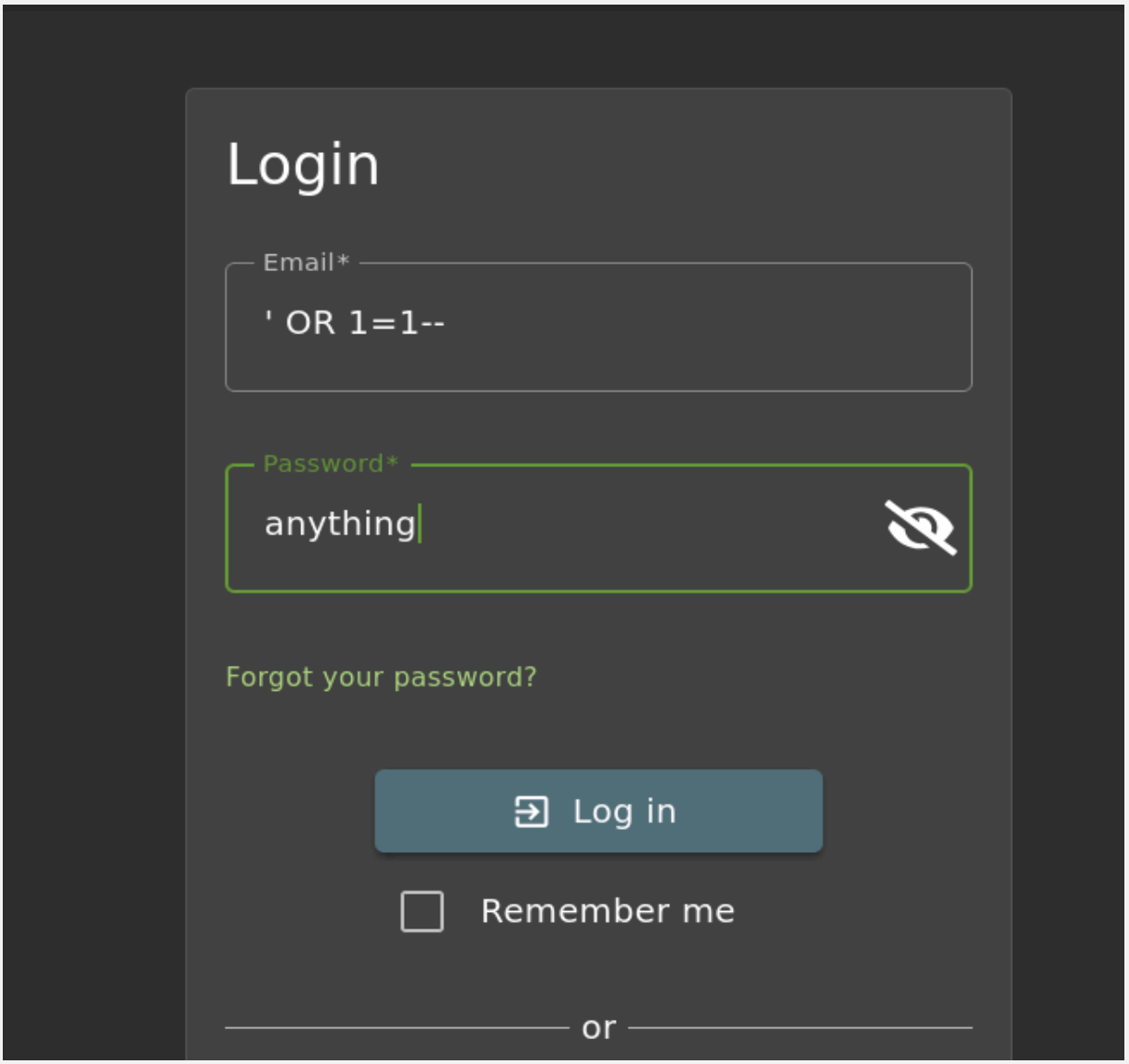
Remediation

Replace the custom implementation with a standard library (e.g., PyCryptodome) using RSA-OAEP. Never implement cryptographic primitives from scratch. Perform regular code reviews with cryptography expertise.

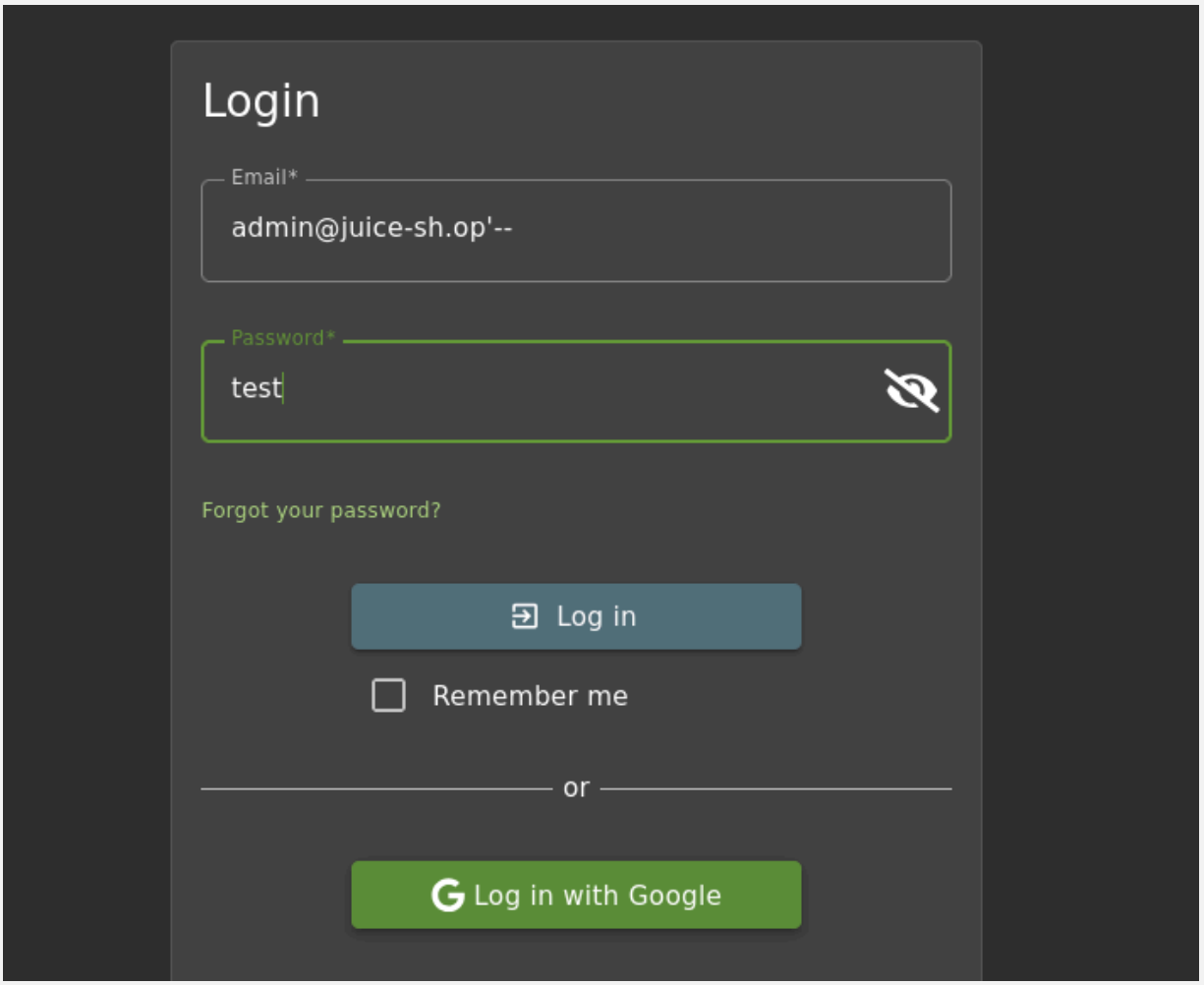
Finding WA-004a: SQL Injection — Authentication Bypass

Severity	Critical
Description	The login form is vulnerable to SQL injection. The email field input is directly concatenated into a SQL query without sanitization or parameterization. By submitting the payload \' OR 1=1-- as the email, the query becomes always-true, and the database returns the first user record (the administrator). This granted full administrative access to the application, including the user management dashboard accessible at /#/administration.
System	http://localhost:3000/#/login
Tools Used	Browser, Burp Suite
References	CWE-89: SQL Injection; OWASP A03:2021 – Injection

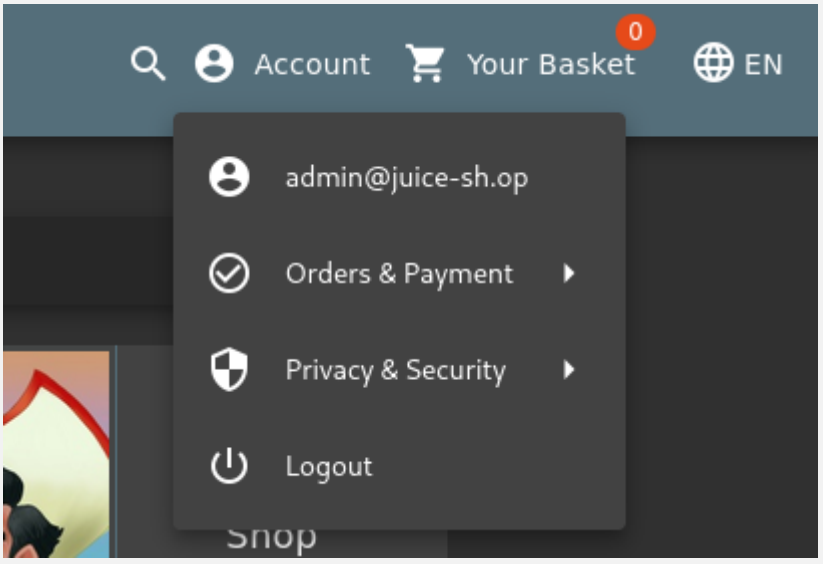
Evidence



Evidence of using previously found admin address



Result Admin Login via SQLI



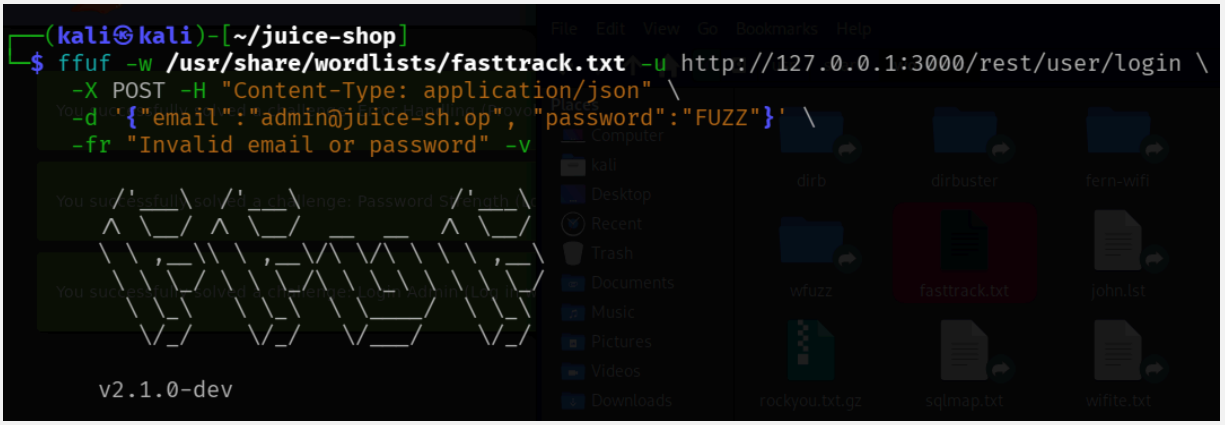
Remediation

Replace all dynamic SQL queries with parameterized statements (prepared statements). Implement server-side input validation. Enforce account lockout policies after repeated failed attempts.

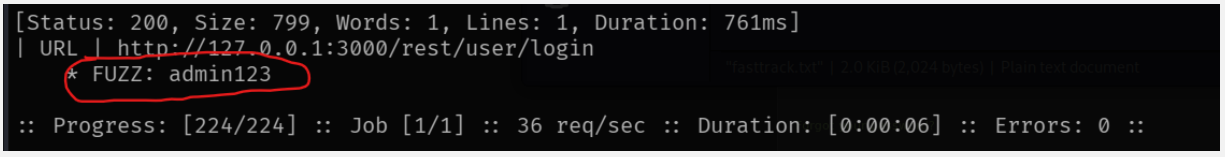
Finding WA-004b: Insufficient Brute-Force Protection on Login Endpoint

Severity	High
Description	The /rest/user/login endpoint enforces no rate limiting, account lockout, or CAPTCHA mechanism. An attacker can submit an unlimited number of password guesses against a known account. Using ffufwith the fasttrack.txt wordlist against the known administrator email (admin@juice-sh.op), the correct password was recovered in seconds. The only server-side control is a string match returning "Invalid email or password", which ffuf's -fr flag trivially filters out to isolate successful responses.
System	http://127.0.0.1:3000/rest/user/login
Tools Used	ffuf, fasttrack.txt wordlis
References	CWE-307: Improper Restriction of Excessive Authentication Attempts; OWASP A07:2021 – Identification and Authentication Failures

Bruteforce Attack command



Evidence of found Admin Password



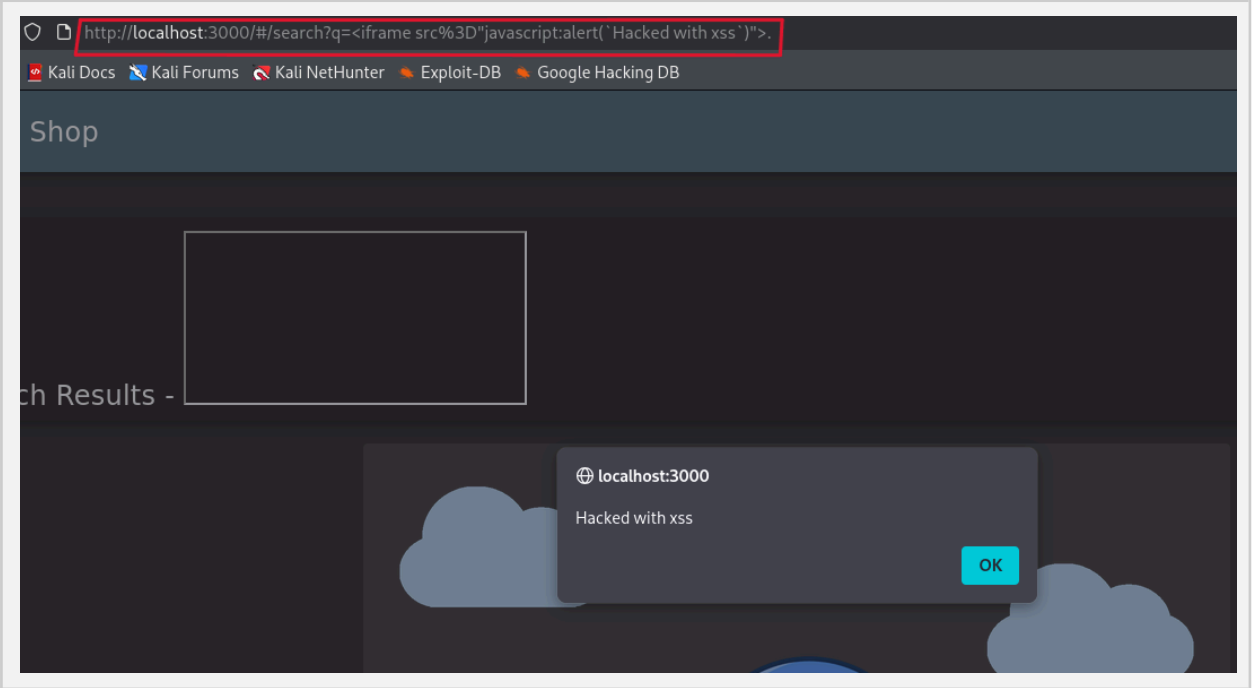
Remediation

Implement express-rate-limit to throttle login attempts per IP (e.g., max 10 requests/minute). Add exponential backoff or temporary account lockout after 5 failed attempts. Log and alert on repeated failures. Consider CAPTCHA for repeated failures from the same IP.

Finding WA-005: Reflected Cross-Site Scripting (XSS) via Search Parameter

Severity	High
Description	The search functionality embeds the value of the q URL parameter directly into the DOM without output encoding. An attacker can craft a malicious URL that, when visited by a logged-in user, executes arbitrary JavaScript in the victim's browser context. The payload <iframe src="javascript:alert('xss')"> successfully triggers execution. Note: the standard <script> tag is filtered by Angular, but iframe and img onerror payloads bypass this sanitization.
System	http://localhost:3000/#/search?q=<iframe src="javascript:alert('xss')">
Tools Used	Browser, Burp Suite
References	CWE-79: Improper Neutralization of Input During Web Page Generation; OWASP A03:2021 – Injection

Evidence



Remediation

Apply context-aware output encoding using a trusted library before rendering user-supplied data. Implement a strict Content Security Policy (CSP) that disallows inline scripts and javascript: URIs. Review all Angular trust bypasses (bypassSecurityTrustHtml) in the codebase.

Finding WA-006: Application-Level Denial of Service via Resource Exhaustion

Severity	High
Description	During automated directory enumeration with ffuf, the Node.js backend exhausted its V8 heap memory and crashed with a FATAL ERROR: Ineffective mark-compacts near heap limit. The application processes each request through its wildcard routing logic, consuming memory for each connection. A high-volume scan (default threading) was sufficient to trigger a complete service outage. The application has no rate limiting or request throttling mechanism.
System	http://localhost:3000(all routes)
Tools Used	ffuf (high-thread scan)
References	CWE-400: Uncontrolled Resource Consumption; OWASP A05:2021 – Security Misconfiguration

Evidence



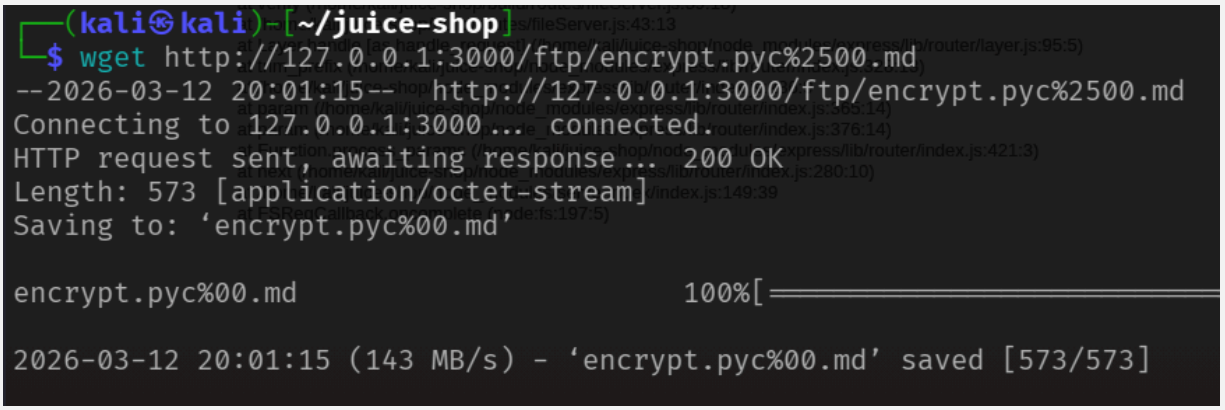
Remediation

Implement the express-rate-limit middleware to throttle requests per IP. Use a process manager such as PM2 to automatically restart the service on crash. Add memory monitoring and alerting. Consider increasing the Node.js heap limit as a secondary measure only.

Finding WA-007: File Download Filter Bypass via Null Byte Injection

Severity	Medium
Description	The /ftp endpoint implements a whitelist filter allowing only .md and .pdf file downloads. This control is bypassable using a double-encoded Null Byte in the URL: appending %2500.md to a filename causes the application layer to see a .md extension and allow the request, while the underlying filesystem layer interprets the %2500 sequence as a Null Byte, truncating the filename at that point. This allowed binary files such as incident-support.kdbx and encrypt.pyc to be downloaded.
System	http://localhost:3000/ftp/*
Tools Used	Browser, curl
References	CWE-626: Null Byte Interaction Error; OWASP A01:2021 – Broken Access Control

Example of bypassing the download control with "%2500.md"



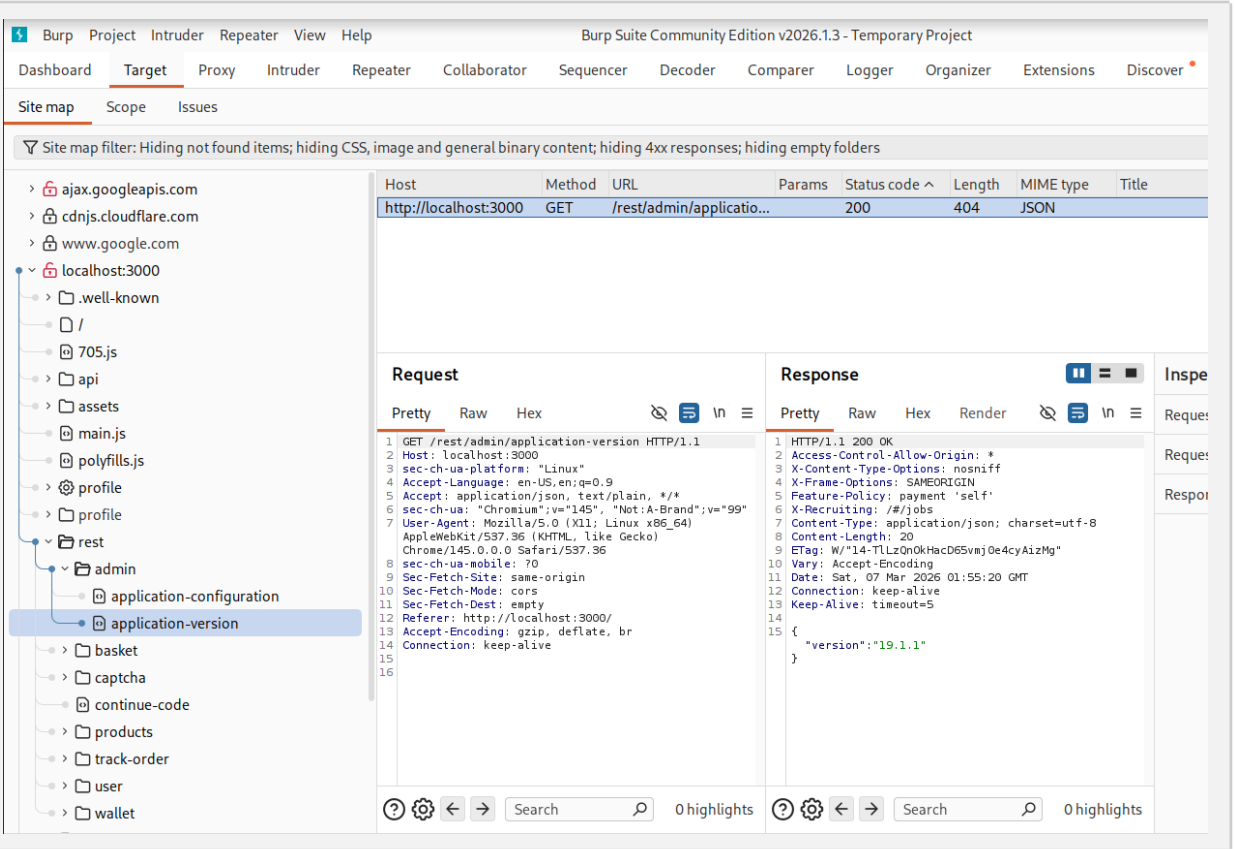
Remediation

Sanitize all file download request parameters to strip Null Bytes (%00, %2500) and path traversal sequences. Reject requests with unexpected characters. Serve files through a controlled backend endpoint rather than exposing the filesystem directly.

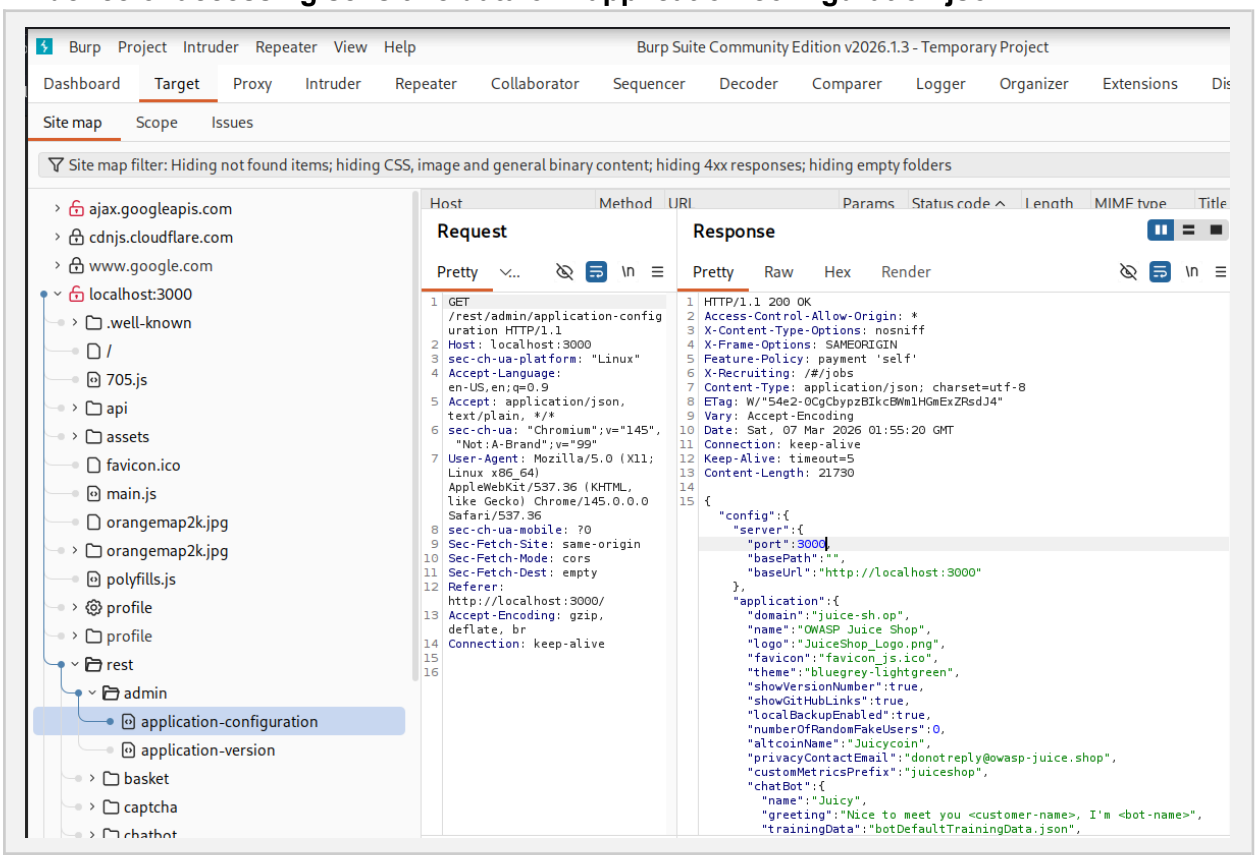
Finding WA-008: Unauthenticated Admin API Access & Information Disclosure

Severity	Medium
Description	The /rest/admin/ API namespace is accessible without a valid session token. The endpoint /rest/admin/application-configuration returns a 200 OK response containing internal server configuration including port number (3000), base URL, backup status (localBackupEnabled: true), and internal filenames. The endpoint /rest/admin/application-version discloses the exact application version (19.1.1), enabling targeted vulnerability research.
System	http://localhost:3000/rest/admin/application-configuration, /rest/admin/application-version
Tools Used	Burp Suite Site Map, Browser
References	CWE-200: Exposure of Sensitive Information; CWE-284: Improper Access Control; OWASP A01:2021

Evidence of accessing sensitive data on “application-version.json”



Evidence of accessing sensitive data on “application-configuration.json”



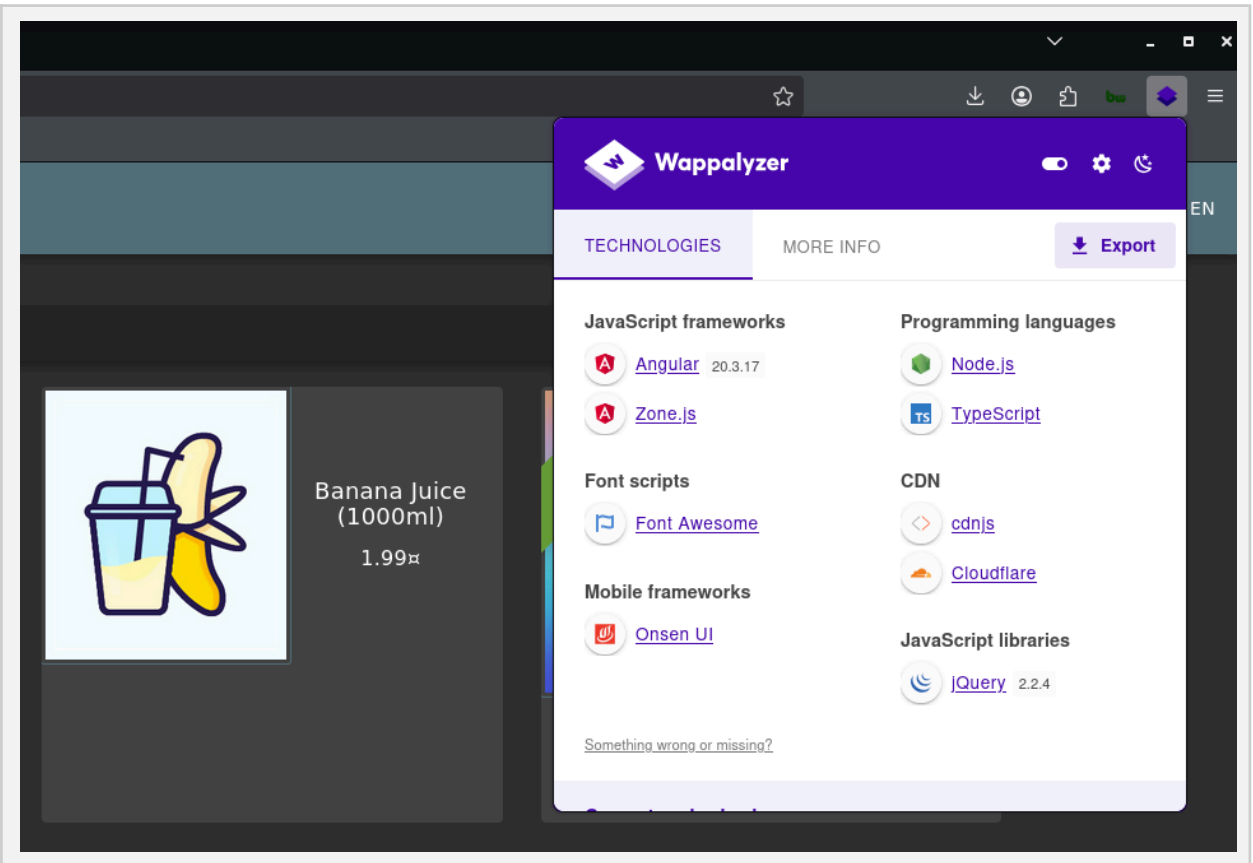
Remediation

Restrict all /rest/admin/* endpoints to authenticated and authorized sessions. Return 401 Unauthorized for unauthenticated requests. Avoid exposing internal configuration details via APIs, even to authenticated users.

Finding WA-009: Use of Components with Known Vulnerabilities

Severity	Low
Description	Technology fingerprinting via Wappalyzer identified jQuery version 2.2.4, which is significantly outdated. This version is known to be affected by CVE-2015-9251 (XSS via cross-domain Ajax requests). Additionally, Nikto analysis confirmed the absence of several HTTP security headers including X-Content-Type-Options, and identified an overly permissive CORS policy (Access-Control-Allow-Origin: *).
System	All pages (global jQuery include)
Tools Used	Wappalyzer, Nikto
References	OWASP A06:2021 – Vulnerable and Outdated Components; CVE-2015-9251

Evidence



Remediation

Upgrade jQuery to the latest stable version. Add HTTP security headers: X-Content-Type-Options: nosniff, X-Frame-Options: DENY, and a strict Content-Security-Policy. Restrict CORS to specific trusted origins.

6. Conclusion

The OWASP Juice Shop assessment demonstrated a broad range of web application security vulnerabilities spanning seven OWASP Top 10 categories. The most critical issue is the chain of vulnerabilities that allows a fully unauthenticated attacker to exfiltrate sensitive files, decrypt internal communications, and gain administrative control of the application through SQL injection and credential brute-forcing.

The primary root causes identified across findings are:

- Insufficient input validation and lack of parameterized queries.
- Absence of authentication and authorization controls on sensitive endpoints.
- Lack of brute-force protection and rate limiting on authentication endpoints.
- Improper configuration of XML parsers and directory access permissions.
- Use of outdated third-party dependencies and custom cryptographic implementations.

It is strongly recommended that the development team address Critical and High severity findings as an immediate priority before any production deployment. A follow-up assessment should be conducted after remediation to verify the effectiveness of applied fixes.